

Air-Gapped Agentic AI Workbench

System Architecture Reference & Presentation Guide — 8 Core Topics

Architecture: Air-Gapped Multi-Model	GPU Telemetry: RTX 5050 (8 GB)	Verification: 48/48 Tests Pass
--------------------------------------	--------------------------------	--------------------------------

Topic 1: Overview & System Architecture

High-level design, layer breakdown, and air-gapped host telemetry

- 1 **Air-Gapped Architecture:** The workbench is 100% self-contained with zero external internet dependencies; all model inference runs on local hardware.
- 2 **Layered System Pipeline:** Structured into 6 distinct modular layers: Frontend UIs → FastAPI REST Control Plane → Task Router → 5-Phase Agent Loop → Secure Sandbox & RAG → Local Ollama Inference.
- 3 **Dual Frontend Options:** Features a Streamlit interactive workbench (frontend/app.py) for rapid prototyping and a modern React/Vite web application (frontend-web/) for production use.
- 4 **Dynamic Host Hardware Telemetry:** Queries nvidia-smi at runtime to automatically detect the current PC's GPU (NVIDIA GeForce RTX 5050 Laptop GPU, 8 GB VRAM) without hardcoded limits.
- 5 **Comprehensive Test Verification:** Validated against 48 automated unit tests verifying all 6 Definition of Done (DoD) operational criteria.
- 6 **Active Egress Isolation:** Uses continuous host interface sampling to guarantee zero external byte leakage during complex multi-step task execution.

Topic 2: Control Plane & REST API Layer

FastAPI engine, endpoint registry, and human-in-the-loop gates

- 1 **FastAPI Backend Engine:** Central orchestrator (orchestrator/main.py) running on port 8000 serving 21 structured REST endpoints across 6 functional groups.
- 2 **Asynchronous Task Submission:** POST /v1/task receives prompts and workspace file UUIDs, returning a task ID immediately while executing the agent loop in background threads.
- 3 **Human-in-the-Loop Approval Gate:** POST /v1/task/{id}/approve enables human operators to review multi-step execution plans before final document generation.
- 4 **Zero-Redeploy Model Management:** POST /v1/models/register allows registering new LLM tags and VRAM budgets dynamically without restarting the server.
- 5 **Full Workspace File Lifecycle:** Provides dedicated endpoints (/v1/workspace/*) for uploading raw attachments, listing active files, and downloading generated office deliverables.
- 6 **Knowledge Base & Audit APIs:** Dedicated routes for document ingestion (/v1/knowledge-base/ingest) and fetching tamper-evident event logs (/v1/audit/logs).

Topic 3: Task Router & Intelligent Model Selector

Pattern classification waterfall, confidence scores, and capability matching

- 1 **Automated Task Classification:** Evaluates prompts and attachment MIME types against high-precision regex pattern matrices in `classifier.py`.
 -
- 2 **Priority Waterfall Matrix:** Routes tasks across 8 task types with calibrated confidence scores: Vision OCR (0.95), Multi-Step Plan (0.95), Code Gen (0.92), Spreadsheet Calc (0.90), RAG Search (0.89), Doc Draft (0.88), Summarization (0.85), and General QA (0.80).
 -
- 3 **VRAM-Aware Model Selection:** `ModelRegistry.route_task()` in `selector.py` matches required task capabilities against local model VRAM footprints.
 -
- 4 **Multi-Tool Detection:** Automatically elevates complex prompts involving multiple tools (e.g., code + doc draft) to the `multi_step_plan` workflow.
 -
- 5 **Automatic Model Fallback:** Gracefully routes tasks to `llama3.1:8b` if a specialized model is missing or unpulled in Ollama.
 -
- 6 **Sub-Millisecond Routing Latency:** Runs classification locally in under 1ms before invoking any LLM inference.
 -

Topic 4: Agent Execution Loop & State Machine

Autonomous ReAct lifecycle, SQLite state persistence, and self-correction

- 1 **5-Phase Autonomous Lifecycle:** Implements `Plan` → `Act` → `Observe` → `Reflect` → `Finalize` inside the `AgentExecutor` state machine (`graph.py`).
 -
- 2 **Phase 1 (Plan):** Deconstructs user intent into up to 10 ordered tool steps with explicit argument schema validation (`_generate_plan()`).
 -
- 3 **Phase 2 & 3 (Act & Observe):** Dispatches tool calls to local python modules, captures stdout/deliverables, and appends observations to context.
 -
- 4 **Phase 4 (Reflect & Self-Correct):** Catches execution failures (e.g., syntax errors) and automatically retries with adjusted arguments up to 2 times (`_self_correct_args()`).
 -
- 5 **Phase 5 (Finalize):** Synthesizes all step observations into a coherent markdown answer and lists generated deliverable paths.
 -
- 6 **SQLite Task State Checkpointing:** Saves complete task snapshots (`StepRecord` and `TaskState`) to `data/agent_tasks.db` after every single step for full crash recovery.
 -

Topic 5: Secure Sandbox & Document Generators

Linux Bubblewrap isolation, path allowlisting, and enterprise document creation

- 1 **Bubblewrap Kernel Isolation:** Executes untrusted Python scripts via Linux bwrap using kernel namespaces
• (`--unshare-net`, `--unshare-ipc`, `--unshare-pid`).
- 2 **Zero Network Egress Enforced:** Network namespace is completely removed; any socket connection attempt inside the sandbox immediately fails.
- 3 **Workspace Containment:** `_safe_path()` in `files.py` enforces that all read/write operations stay strictly within the assigned workspace directory, blocking path traversal.
- 4 **Enterprise Word Documents (.docx):** `docgen.py` creates formal 5-section Approval Notes (Header, Background, Findings Table, Recommendations, Sign-off) with Navy corporate styling.
- 5 **Audited Excel Spreadsheets (.xlsx):** `spreadsheet.py` uses `openpyxl` to generate spreadsheets with live calculation formulas (`=SUM`, `=AVERAGE`, `=MAX`) and zebra row formatting.
- 6 **Executive PowerPoint Decks (.pptx):** Generates dark-themed presentation slides with structured titles and content cards.

Topic 6: Data Ingestion & Hybrid RAG Retrieval

MIME detection, scanned PDF OCR, ChromaDB indexing, and BM25 hybrid search

- 1 **Header-Based MIME Detection:** Inspects raw file magic bytes in `loaders.py` to reliably route PDFs, DOCX files, and raw images.
- 2 **Scanned PDF Auto-Classification:** Automatically detects scanned PDFs (<50 chars extracted + image blocks present) and routes pages through the vision OCR pipeline.
- 3 **Page-by-Page Vision OCR:** `VisionOCRPipeline` renders PDF pages as 200dpi PNGs, base64 encodes them, and extracts text via the local `moondream` vision model.
- 4 **Hybrid Search Formula ($\alpha = 0.7$):** Combines ChromaDB vector similarity (0.7 weight) with in-memory BM25 keyword matching (0.3 weight) for optimal document retrieval accuracy.
- 5 **Exact Citation Tracking:** Every retrieved text chunk returns doc ID, filename, page number, and chunk index for verifiable audit citations.
- 6 **Deterministic Embedding Fallback:** If the Ollama embedding model (`nomiic-embed-text`) is unavailable, falls back to SHA-256 text hashing to produce stable float vectors.

Topic 7: Air-Gap Security & Audit Logging

Hardware egress monitoring, dual-write audit logs, and DoD compliance

- 1 **Real-Time Egress Monitoring:** NetworkMonitor (monitor.py) samples network interface counters via psutil every second, ignoring loopback interfaces.
- 2 **Zero-Egress Verdict:** Flags system status as SECURE_AIR_GAPPED when external send rate is below 50 B/s and total external bytes sent equals zero.
- 3 **Active Socket Inspection:** Continuously audits open TCP/UDP sockets to flag and log any external IP connections.
- 4 **Dual-Write Tamper-Evident Logging:** AuditLogger (logger.py) writes all model calls, tool executions, and approvals to both SQLite (audit.db) and JSONL (audit_log.jsonl).
- 5 **DoD 6-Point Compliance:** Passes all 6 Definition of Done criteria (Model Routing, Approval Note .docx, Sandbox Code Exec, Multimodal Schema, Air-Gap Proof, Zero-Redeploy Model Add).
- 6 **Terminal Egress Verification:** Includes pre-configured command (sudo tcpdump -i any not host 127.0.0.1 -n -c 20) for live third-party egress auditing.

Topic 8: Local LLM Models & VRAM Allocation

Ollama model inventory, VRAM budgets, and fallback invocation chains

- 1 **100% Local Inference Engine:** All language and vision models run on-device via Ollama at 127.0.0.1:11434 with zero cloud connectivity.
- 2 **qwen2.5-coder:7b (4.7 GB VRAM):** Primary coding model specialized for Python script authoring, code debugging, and Excel formula generation.
- 3 **llama3.1:8b (4.9 GB VRAM):** Primary reasoning engine for general Q&A, document drafting, multi-step plan synthesis, and RAG search response generation.
- 4 **deepseek-r1:7b (4.7 GB VRAM):** Chain-of-thought mathematical reasoning model for engineering calculations and step-by-step logic traces.
- 5 **moondream:latest (2.0 GB VRAM):** Lightweight 1.7GB vision model for document image OCR, diagram inspection, and visual Q&A.
- 6 **nomic-embed-text (0.3 GB VRAM):** Compact embedding model generating 768-dimensional dense vectors for ChromaDB knowledge base indexing.